



**Instituto Tecnológico Superior De
Rio verde**

Alumno: Luis Alejandro Domínguez Ramírez

Materia: Programación orientada a objetos

Profesor: ING. José de Jesús Collazo Reyes

Grupo: 2-A

Temas: Herencia en POO

Fecha: 28/04/2026

INTRODUCCION

Una clase abstracta es una clase que se utiliza como base para otras clases dentro de la programación orientada a objetos. Este tipo de clase no puede crear objetos directamente y sirve para definir atributos y métodos generales que compartirán las clases hijas. Además, puede contener métodos virtuales puros que obligan a las clases derivadas a implementar ciertos comportamientos específicos.

Una interfaz es una estructura que únicamente contiene métodos virtuales puros. Su función es establecer acciones o responsabilidades que las clases deben implementar obligatoriamente. Las interfaces ayudan a que diferentes clases puedan compartir comportamientos comunes aunque funcionen de manera distinta.

La diferencia principal entre una clase abstracta y una interfaz es que la clase abstracta puede contener atributos, métodos normales y métodos con implementación, mientras que la interfaz solamente define métodos obligatorios sin implementar código ni almacenar atributos. La clase abstracta funciona como una plantilla general y la interfaz como un contrato de comportamiento.

Estos conceptos son importantes en la programación orientada a objetos porque permiten desarrollar programas más organizados, reutilizables y fáciles de mantener. También ayudan a aplicar herencia y polimorfismo, permitiendo que diferentes objetos puedan trabajar de manera flexible utilizando una misma estructura base.

DESARROLLO

Explicación del sistema

El sistema desarrollado tiene como objetivo administrar distintos tipos de empleados dentro de una empresa tecnológica. El programa permite registrar empleados, mostrar su información, ejecutar actividades y generar reportes dependiendo del tipo de trabajador.

Para lograrlo se utilizó programación orientada a objetos en C++, implementando clases abstractas, interfaces, herencia y polimorfismo. Todos los empleados comparten características generales como nombre, edad y salario, pero cada uno realiza tareas diferentes según su función dentro de la empresa.

El sistema cuenta con tres tipos de empleados:

- Programador
- Administrador de Red
- Soporte Técnico

Cada uno tiene comportamientos específicos implementados mediante sobrescritura de métodos.

DISEÑO DEL SISTEMA

La clase principal del sistema es la clase abstracta Empleado, la cual contiene los atributos y métodos generales que compartirán todos los empleados.

Clase abstracta: Empleado

Atributos:

- nombre
- edad
- salario

Métodos:

- mostrarInformacion()
- trabajar()
- realizarActividad()

El método trabajar () fue declarado como virtual puro para obligar a las clases hijas a implementar su propia versión.

Interfaz: Responsable

La interfaz IResponsable contiene el método:

- generarReporte()

Esta interfaz obliga a todas las clases derivadas a generar un reporte relacionado con sus actividades.

Clases derivadas

Programador

Hereda de Empleado e implementa IResponsable.

Funciones:

- desarrollar código
- generar reportes de avances del sistema

AdministradorRed

Hereda de Empleado e implementa IResponsable.

Funciones:

- monitorear servidores y red
- generar reportes del estado de la red

SoporteTecnico

Hereda de Empleado e implementa IResponsable.

Funciones:

- atender incidencias
- generar reportes de tickets atendidos

Relación entre clases e interfaces

La relación principal del sistema se basa en la herencia y la implementación de interfaces.

La clase Empleado funciona como clase base abstracta de la cual heredan las demás clases del sistema. Esto permite reutilizar atributos y métodos comunes para todos los empleados.

Las clases Programador, AdministradorRed y SoporteTecnico implementan la interfaz IResponsable, lo que las obliga a desarrollar el método generarReporte ().

Además, el sistema utiliza polimorfismo mediante un vector de punteros tipo Empleado, permitiendo manejar diferentes tipos de empleados utilizando una misma estructura base. Gracias a esto, cada objeto responde de manera distinta al ejecutar métodos como trabajar ().

CODIGO FUENTE

Empleado.h

```
#ifndef EMPLEADO_H
#define EMPLEADO_H

#include <iostream>
#include <string>

using namespace std;

// Interfaz
class IResponsable {
public:
    virtual void generarReporte() = 0;
};

class Empleado {
protected:
    string nombre;
    int edad;
    float salario;
public:
    Empleado(string n, int e, float s) {
        nombre = n;
        edad = e;
        salario = s;
    }

    virtual void mostrarInformacion() {
        cout << "Nombre: " << nombre << endl;
        cout << "Edad: " << edad << endl;
        cout << "Salario: $" << salario << endl;
    }

    virtual void trabajar() = 0;

    virtual void realizarActividad() {
```

En este archivo se creó la interfaz IResponsable y la clase abstracta Empleado. La interfaz obliga a las clases derivadas a implementar el método generarReporte (). Después se define la clase Empleado, donde se almacenan los atributos generales como nombre, edad y salario.

También se incluyen métodos como mostrarInformacion () para enseñar los datos del empleado y realizarActividad () para mostrar una acción general. El método trabajar () fue declarado como virtual puro, lo que convierte a Empleado en una clase abstracta y obliga a las clases hijas a crear su propia versión de este método.

Además, se agregó un destructor virtual para liberar memoria correctamente cuando se utilizan punteros y polimorfismo.

Empleado.cpp

```
1 #include "Empleado.h"
2
3 class Programador : public Empleado, public IResponsable {
4 public:
5     Programador(string n, int e, float s)
6         : Empleado(n, e, s) {}
7
8     void trabajar() override {
9         cout << nombre << " esta desarrollando codigo." << endl;
10    }
11
12    void generarReporte() override {
13        cout << "Reporte: avances del sistema generados." << endl;
14    }
15 };
16
17
18 class AdministradorRed : public Empleado, public IResponsable {
19 public:
20     AdministradorRed(string n, int e, float s)
21         : Empleado(n, e, s) {}
22
23     void trabajar() override {
24         cout << nombre << " esta monitoreando servidores y red." << endl;
25    }
26
27    void generarReporte() override {
28        cout << "Reporte: estado de la red generado." << endl;
29    }
30 };
31
32
33 class SoporteTecnico : public Empleado, public IResponsable {
34 public:
35     SoporteTecnico(string n, int e, float s)
36         : Empleado(n, e, s) {}
37 }
```

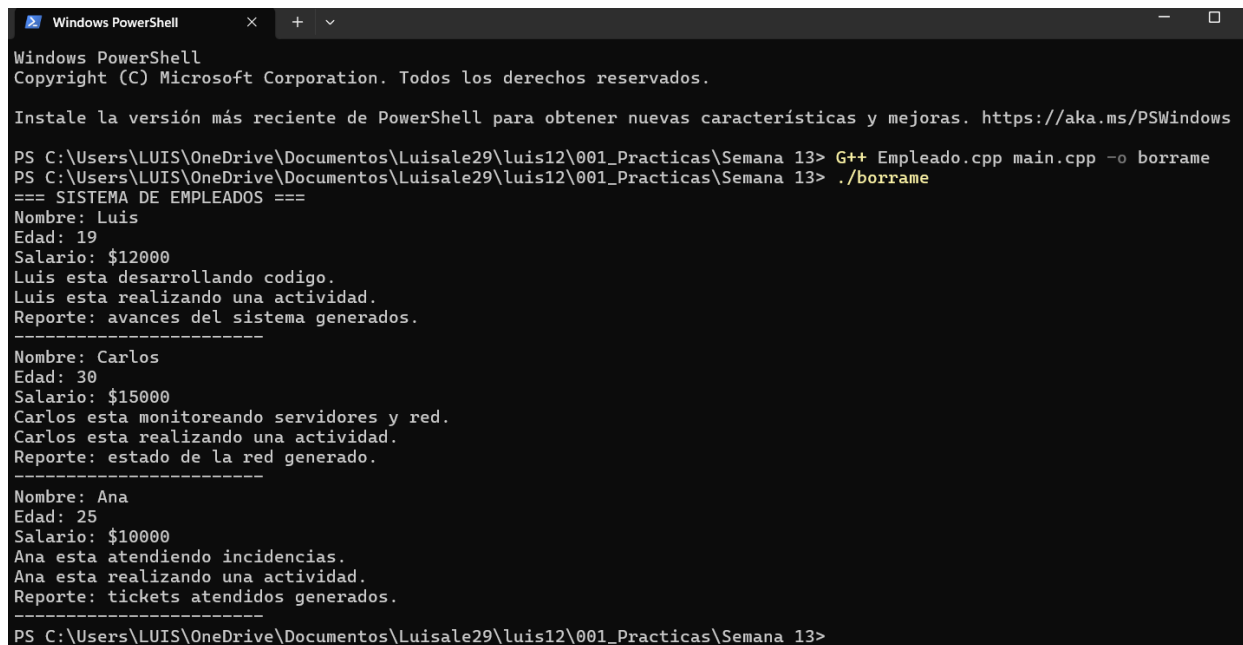
En el archivo Empleado.cpp se implementaron las clases derivadas Programador, AdministradorRed y SoporteTecnico. Cada clase hereda de Empleado e implementa la interfaz IResponsable, lo que permite que cada empleado tenga comportamientos específicos dependiendo de su función dentro de la empresa. Por ejemplo, el programador desarrolla código, el administrador de red monitorea servidores y el soporte técnico atiende incidencias. Además, cada clase genera reportes distintos mediante el método generarReporte (). En este archivo se aplica la herencia y el polimorfismo, ya que todas las clases utilizan la misma estructura base pero responden de manera diferente.

main.cpp

```
main.cpp 2 @main()
1  #include <iostream>
2  #include <vector>
3  #include "Empleado.cpp"
4
5  using namespace std;
6
7  int main() {
8
9      vector<Empleado*> empleados;
10
11      empleados.push_back(new Programador("Luis", 19, 12000));
12      empleados.push_back(new AdministradorRed("Carlos", 30, 15000));
13      empleados.push_back(new SoporteTecnico("Ana", 25, 10000));
14
15      cout << "=== SISTEMA DE EMPLEADOS ===" << endl;
16
17      for (Empleado* empleado : empleados) {
18
19          empleado->mostrarInformacion();
20          empleado->trabajar();
21          empleado->realizarActividad();
22
23          IResponsable* responsable =
24              dynamic_cast<IResponsable*>(empleado);
25
26          if (responsable != nullptr) {
27              responsable->generarReporte();
28          }
29
30          cout << "-----" << endl;
31      }
32
33      for (Empleado* empleado : empleados) {
34          delete empleado;
35      }
36
37      return 0;
```

En el archivo main.cpp se desarrolló la parte principal del programa. Aquí se creó un vector de punteros tipo Empleado para almacenar diferentes tipos de empleados utilizando polimorfismo. Después se agregaron objetos de las clases derivadas y se recorrieron mediante un ciclo para ejecutar acciones como mostrar información, trabajar, realizar actividades y generar reportes. Gracias al polimorfismo, cada objeto respondió de manera distinta dependiendo de su tipo, aunque todos fueron manejados utilizando la clase base Empleado. Finalmente, se liberó la memoria utilizando delete para evitar fugas de memoria en el programa.

CAPTURAS EJECUCION



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\LUIS\OneDrive\Documentos\Luisale29\luis12\001_Practicas\Semana 13> G++ Empleado.cpp main.cpp -o borrame
PS C:\Users\LUIS\OneDrive\Documentos\Luisale29\luis12\001_Practicas\Semana 13> ./borrame
=== SISTEMA DE EMPLEADOS ===
Nombre: Luis
Edad: 19
Salario: $12000
Luis esta desarrollando codigo.
Luis esta realizando una actividad.
Reporte: avances del sistema generados.
-----
Nombre: Carlos
Edad: 30
Salario: $15000
Carlos esta monitoreando servidores y red.
Carlos esta realizando una actividad.
Reporte: estado de la red generado.
-----
Nombre: Ana
Edad: 25
Salario: $10000
Ana esta atendiendo incidencias.
Ana esta realizando una actividad.
Reporte: tickets atendidos generados.
-----
PS C:\Users\LUIS\OneDrive\Documentos\Luisale29\luis12\001_Practicas\Semana 13>
```

En esta ejecución del programa se puede observar el funcionamiento correcto del sistema de empleados desarrollado en C++. El programa muestra la información de cada empleado, incluyendo su nombre, edad y salario. Después, cada objeto ejecuta el método trabajar (), donde se demuestra el polimorfismo, ya que cada empleado realiza una actividad diferente según su tipo. Finalmente, cada clase genera su propio reporte mediante la interfaz IResponsable, comprobando el uso de herencia, clases abstractas e interfaces dentro del sistema.

EXPLICACION TECNICA

La clase abstracta se implementó mediante la creación de la clase Empleado, la cual contiene atributos generales como nombre, edad y salario. Además, se declaró el método trabajar () como virtual puro utilizando virtual void trabajar() = 0;, lo que obliga a las clases derivadas a implementar su propia versión de este método. Gracias a esto, la clase Empleado funciona como una base común para todos los tipos de empleados.

La interfaz se implementó mediante la clase IResponsable, la cual contiene únicamente el método virtual puro generarReporte (). Las clases derivadas implementan esta interfaz para poder generar reportes específicos según las actividades de cada empleado.

El uso de interfaces tiene varias ventajas, ya que permite definir responsabilidades comunes entre distintas clases sin importar cómo funcionen internamente. También ayuda a mantener el código más organizado, flexible y fácil de expandir en el futuro.

El polimorfismo ayuda en este sistema porque permite trabajar con diferentes tipos de empleados utilizando una misma estructura base. Gracias al uso de un vector de punteros tipo Empleado, el programa puede recorrer distintos objetos y ejecutar métodos como trabajar () de manera diferente dependiendo del tipo de empleado, haciendo el sistema más dinámico y reutilizable.

CONCLUSION

Durante esta práctica se presentaron algunas dificultades al momento de comprender cómo funcionan las clases abstractas, las interfaces y el polimorfismo dentro de un mismo sistema. También fue complicado organizar correctamente las clases y entender cómo utilizar los métodos virtuales puros para que cada empleado realizara actividades diferentes. Sin embargo, con la práctica fue posible comprender mejor la relación entre herencia, interfaces y polimorfismo en C++.

Gracias a esta actividad aprendí a desarrollar un sistema utilizando programación orientada a objetos de una manera más organizada y reutilizable. También aprendí cómo crear clases abstractas, implementar interfaces y utilizar polimorfismo para manejar diferentes tipos de objetos utilizando una misma estructura base.

Este tipo de conocimientos puede aplicarse en una empresa real para desarrollar sistemas administrativos, programas de recursos humanos o plataformas donde existan diferentes tipos de usuarios con funciones específicas. Además, permite crear programas más flexibles, fáciles de mantener y preparados para agregar nuevas funciones en el futuro.